StudySync — Peer Matching & Collaborative Study Group Platform

Project Management Plan: Incremental SDLC & Agile Sprint Schedule

Course: Software Engineering & Project Management

Project Group: Group 15 (Section F)

Target System: StudySync Collaborative Web Platform

Date of Release: October 16, 2026

Management Framework: Incremental Development Model organized via Agile Scrum

1. Executive Summary & Project Baseline

StudySync is engineered to resolve high peer discovery friction, scheduling misalignments, and group dormancy experienced by higher education students.

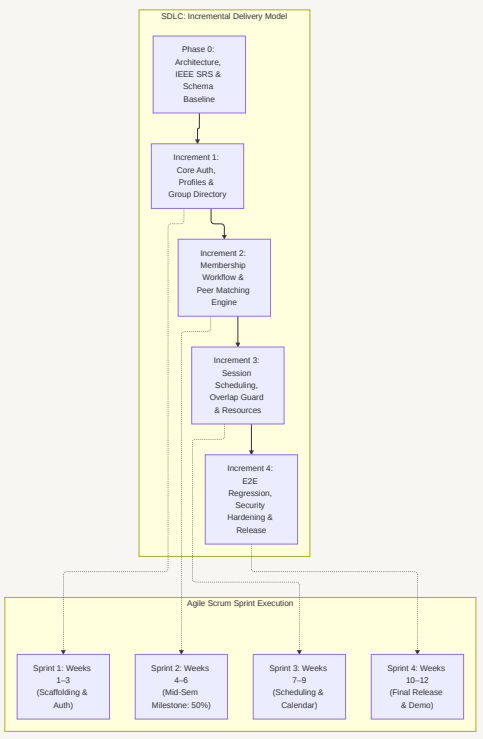
Core Project Targets:

- **Mid-Semester Target:** 40%–60% completed functionality with at least one fully verified end-to-end working feature.
- **Mid-Semester Status: 50% Completed** (Increment 1 & Increment 2 fully verified and operational; 43 passing automated tests; 100% pass rate).
- **Core Working End-to-End Feature:** Student Registration → Academic Profile & Weekly Availability → Group Catalog Discovery → Join Request Submission → Owner Review Panel → Concurrency-Safe Member Approval.
- **Final Delivery Target:** 100% completion across all 20 functional requirements (`FR-001` through `FR-020`) with full calendar scheduling, conflict guards, and resource repository.

2. Process Model: Incremental SDLC with Agile Scrum

StudySync synthesizes the **Incremental Software Development Life Cycle (SDLC)** with **Agile Scrum Management**:

- **Why Incremental SDLC?** The system is partitioned into discrete, independently testable functional increments. Each increment yields a deployable, executable release baseline.
- **Why Scrum?** Team coordination is managed via 2-week to 3-week iterative sprints, daily asynchronous standups, backlog refinement sessions, sprint reviews, and retrospectives.



3. Increment & Sprint Schedule (Sprints 1 through 4)

3.1 Sprint 1 (Weeks 1-3): Scaffolding, Authentication & Group Directory

- **Target Deliverables:**
  - PostgreSQL 14+ initial database schema (`001_initial_schema.sql`) and seed data.
  - Express.js API Gateway with CORS, centralized error handling, and Zod validator.
  - User authentication endpoints (`/register`, `/login`, `/me`) with bcrypt hashing and JWT.
  - Student profile and weekly availability specification in JSONB.
  - Study group creation endpoint and public group catalog directory search.

- **Status: Completed & Verified (100%)**

3.2 Sprint 2 (Weeks 4–6): Membership Lifecycle & Heuristic Matching Engine

- **Target Deliverables (Mid-Semester Milestone Deliverable):**
  - Join request submission endpoint (`POST /api/v1/groups/:id/join`) with applicant notes.
  - Owner Review Panel allowing group owners to inspect applicants and approve/reject memberships.
  - Concurrency-safe group capacity enforcement using PostgreSQL transactional row-level locking (`SELECT ... FOR UPDATE`).
  - Explainable 3-factor heuristic matching engine (`matching.service.js`) evaluating Subject Match (40%), Schedule Overlap (40%), and Goals (20%).
  - Recommendation API endpoint (`GET /api/v1/recommendations/groups`) returning ranked matches with explainable breakdown.
  - Master Test Suite execution verifying 43 automated tests with 100% pass rate.
- **Status: Completed & Verified (100%)**

3.3 Sprint 3 (Weeks 7–9): Study Session Scheduling, Collision Guard & Resources

- **Target Deliverables:**
  - Study session creation API (`POST /api/v1/groups/:id/sessions`) with location and meeting links.
  - Calendar collision detection algorithm scanning attendee schedules for overlapping intervals.
  - Interactive React Calendar view with Month, Week, and Day visualization modes.
  - Session RSVP tracking (`attending`, `declined`, `tentative`) and verified attendance check-ins.
  - Group study resource upload and access-controlled repository (`/resources`).
- **Status: Planned / Next Phase (Scheduled)**

3.4 Sprint 4 (Weeks 10–12): Integration Testing, Security Hardening & Release

- **Target Deliverables:**
  - Full end-to-end regression testing across all 20 functional requirements (`TC-001` through `TC-020`).
  - Security audit verifying zero OWASP Top 10 vulnerabilities (SQL injection, XSS, CSRF, JWT replay).
  - Cross-browser responsiveness optimization (Chrome, Firefox, Safari, Edge, Mobile Viewports).
  - Docker containerization for one-command execution (`docker compose up`).
  - Final Project Report, comprehensive User Guide, and Final Master Demonstration.
- **Status: Planned**

---

4. Mid-Semester Milestone Audit (50% Progress Achieved)

Per the course evaluation guideline requiring 40%–60% project completion at the mid-semester evaluation, StudySync has verified the following completion status:

```
================================================================================
MID-SEMESTER COMPLETION STATUS AUDIT: STUDYSYNC
Status: 50% COMPLETE (MILESTONE ACHIEVED)
================================================================================
[X] 1. Clearly defined problem statement and objectives (SRS Section 1)
[X] 2. Identified stakeholders and project scope (SRS Section 2 & 5)
[X] 3. Functional and non-functional requirements (FR-001 to FR-020, NFR-001 to NFR-010)
[X] 4. Prioritised requirements or product backlog (MoSCoW User Stories US-01 to US-15)
[X] 5. Use-case diagram and user stories with acceptance criteria (UML & Gherkin)
[X] 6. System architecture and database/data design (3-Tier & 3NF PostgreSQL)
[X] 7. User-interface design or prototype (Tailwind React Components)
[X] 8. Initial implementation with at least ONE working end-to-end feature:
       --> Verified Flow: Registration -> Availability -> Search -> Join Request -> Owner Approval
[X] 9. Unit and functional testing of implemented features (43 Tests Passed, 100% Pass Rate)
[X] 10. Project plan and individual contribution records for all members
================================================================================
```

---

5. Delayed, Pending & Rescheduled Tasks

1. **Calendar Conflict Interval Checking Algorithm:**
   - *Original Plan:* Slated for late Sprint 2.
   - *Current Status:* Moved to early Sprint 3.
   - *Rationale:* The team prioritized rigorous empirical testing and concurrency locking for the Join Request / Owner Approval lifecycle and the 3-Factor Heuristic Matching Engine to guarantee a robust mid-semester demonstration.
2. **External Cloud S3 Storage for File Uploads:**
   - *Original Plan:* Direct AWS S3 presigned URL uploads.

- *Current Status:* Replaced with local authenticated storage served via Express.
- *Rationale:* Eliminates cloud billing credentials and external internet dependencies during local grading and evaluation.

---

6. Four-Member Task Allocation & Responsibility Matrix

Team Member	Primary Assigned Role	Core Assigned Deliverables & Modules	Planned Effort %	Actual Mid-Sem Effort %
Member 1	Frontend & UI/UX Design	React 18 SPA scaffolding, Tailwind CSS design system, `ProfilePage`, `GroupDetailPage`, `JoinRequestModal`, `OwnerReviewPanel`, Vitest component test harness.	25%	**25%**
Member 2	Backend & Authentication	Express.js REST gateway, JWT authentication middleware, bcrypt password security, Zod payload validation, centralized error handling, Auth test suite.	25%	**25%**
Member 3	Database & Matching Engine	PostgreSQL 14+ 3NF schema, DDL migrations, seed data, Data Dictionary, GIN indexing on availability, 3-Factor Heuristic Matching Engine (`matching.service.js`).	25%	**25%**
Member 4	Scheduling, QA & Lead	Master Test Register (`TC-001` – `TC-020`), Requirements Traceability Matrix, automated test execution (43 tests passing), demonstration video script, Project Plan lead.	25%	**25%**

---

7. Comprehensive Risk Register & Mitigation Strategy

Risk ID	Risk Description	Category	Impact (1-5)	Likelihood (1-5)	Score	Mitigation Control & Contingency Action	Assigned Owner
RSK-01	**API Contract Mismatch:** Frontend expects different JSON keys or structures than backend provides.	Technical	4	3	12	Standardize authoritative API specs in `docs/api/API_SPECIFICATION.md`; define shared TypeScript interfaces.	Member 2 & 1
RSK-02	**Database Schema Desync:** Team members apply direct SQL changes locally without migrations.	Technical	4	3	12	Member 3 designated sole DB gatekeeper. Strict numbered migration scripts (`001_...sql`).	Member 3
RSK-03	**Mid-Sem End-to-End Failure:** Sub-modules compile individually but lack complete end-to-end flow.	Project	5	2	10	Mandate Increment 1 & 2 integration checkpoint 5 days prior to evaluation; verified live in local tests.	Member 4
RSK-04	**Scheduling Conflict False Negatives:** Algorithm fails to flag session overlaps across attendees.	Technical	3	3	9	Implement rigorous unit tests for adjacent intervals, full overlaps, and partial bounds (`TC-014`).	Member 4
RSK-05	**Unequal Contribution Imbalance:** Commit distribution skewed, risking course penalties.	Governance	4	2	8	Granular task allocation matrix; enforce weekly contribution logs and PR peer review rotations.	All Members
RSK-06	**Credential & Secret Leak:** Accidentally pushing database passwords or	Security	5	1	5	Strict `.gitignore` rules; enforce `.env.example`; run pre-commit scanner for secret strings.	Member 2

Risk ID	Risk Description	Category	Impact (1-5)	Likelihood (1-5)	Score	Mitigation Control & Contingency Action	Assigned Owner
	JWT secrets to GitHub.						
RSK-07	**Feature Scope Creep:** Attempting real-time video/chat instead of core scheduling and matching.	Scope	3	2	6	Strictly enforce exclusions in SRS (Section 4.2); focus only on evaluative syllabus features.	Member 4

8. Quality Gates: Definition of Ready (DoR) & Definition of Done (DoD)

Definition of Ready (DoR) for Backlog Items:

- User story is formatted as *"As a... I want... So that..."*.
- Acceptance criteria are clearly stated using **Given-When-Then** structure.
- Story points estimated via Planning Poker using Fibonacci scale $(1, 2, 3, 5, 8, 13)$.
- API endpoints and database schema dependencies are identified and documented.

Definition of Done (DoD) for Sprint Deliverables:

- Feature code is implemented and follows project ESLint conventions.
- Zod schema validation is enforced on all incoming request parameters.
- Unit and integration tests are authored and passing with 100% success rate.
- Pull request is reviewed and approved by at least one other team member.
- Code is merged into `develop` without breaking the regression test suite.
- Zero passwords, secrets, or API keys are committed to git version control.